

Proof of Sequence: A Mechanism for FCFS Transaction Ordering in Byzantine Settings

Fermi Labs

Working Draft v0.4

Abstract

First-come-first-served ordering is the central fairness primitive of continuous markets. Blockchains do not natively provide it. A block leader can inspect pending transactions, delay selected flow, insert its own transactions, and choose the intra-block order. This is the structural surface behind frontrunning, back-running, liquidation races, stale-cancel risk, and queue-position manipulation.

We introduce *Proof of Sequence* (PoSq), a sequencing mechanism for FCFS ordering in a single-leader Byzantine setting. PoSq produces a canonical, tick-indexed transaction stream before market-relevant transaction contents are revealed. It composes six mechanisms: (i) per-user trapdoor timelock encryption over RSA groups, (ii) narrow inclusion windows, (iii) signed admission receipts, (iv) a VDF-braided timechain, (v) validator-indistinguishable polling with fixed-envelope size and relay-decoupled fee payment, and (vi) a minimal sequencing-only consensus that finalizes stream prefixes and rotates leaders on fault, liveness failure, or scheduled epoch end.

PoSq enforces certified FIFO ordering relative to the protocol ingress boundary. Admitted transactions execute in receipt order. Reordering, equivocation, invalid VDF transitions, invalid log transitions, and omission of receipted entries are objectively slashable. Selective censorship collapses to bounded random loss: a sequencer that cannot distinguish probes from users cannot target economically valuable transactions without dropping validator probes and triggering rotation. Griefing is possible at small scale; economically beneficial targeted censorship is not.

PoSq is a sequencing layer with a tightly scoped consensus attached. It does not attempt to solve global wall-clock fairness. It produces a canonical transaction order that downstream execution and full consensus layers can verify, execute, and finalize.

Contents

1	Introduction	4
1.1	Contributions	4
2	Design Goals	5
2.1	FIFO Relative to Certified Ingress	5
2.2	No Last-Look	5
2.3	No Profitable Selective Censorship	6
2.4	Continuous Ordering	6
2.5	Objective Accountability	6
3	System Model	6
3.1	Participants	6
3.2	Network Model	7
3.3	Threat Model	7
4	Time, Ticks, and Windows	7
4.1	Inclusion Window	7
4.2	Reveal Delay	8
5	Per-User Trapdoor Timelock Encryption	8
5.1	Construction	8
5.2	Why This Works for PoSq	8
5.3	Reveal Procedure	9
5.4	Threat-Model Implications	9
6	Transaction Format and Metadata Discipline	9
7	PoSq Admission	10
7.1	Commit Submission	10
7.2	Admission Receipt	10
7.3	Signed Rejection	11
7.4	Full-Window Receipt	11
8	Tick Batches and Canonical Order	11
9	Braided VDF Timechain	12
9.1	Construction	12
9.2	Why Braiding Prevents Pre-computation	12
9.3	Verification	13
10	Data Availability	13
11	Validator-Indistinguishable Polling	13
11.1	Probe Generation	13
11.2	Probe Success and Failure	14
11.3	Per-Bucket Polling and Statistical Calibration	14
11.4	Probe Verifiability	14
11.5	Availability Rotation Certificate	15

11.6 Effect of Polling	15
12 Selective Censorship Collapse	15
13 Fraud Proofs	16
13.1 Equivocation Proof	16
13.2 Reorder Proof	16
13.3 Omission Proof	17
13.4 Invalid VDF Transition Proof	17
13.5 Invalid Log Transition Proof	17
13.6 Late Insertion Proof	17
13.7 Invalid Gap Proof	17
14 ContSeq: Minimal Sequencing Consensus	17
14.1 Scope	17
14.2 Validator Set and Honesty Assumption	17
14.3 Finalization	18
14.4 Leader Rotation	18
14.5 Why a Narrow Consensus Is Appropriate	18
15 Sequencer Bonds and Economic Discipline	19
16 Threshold Choice	19
17 Security Analysis	19
17.1 Ordering Binding	19
17.2 No Plaintext-Based Reordering	20
17.3 No Profitable Selective Censorship	20
17.4 Griefing	20
17.5 Censorship After Receipt	20
17.6 Gaps	20
17.7 Adversarial Probes	21
17.8 Sequencer-Affiliated Flow	21
18 Performance Model	21
19 Deployment Rules	21
20 Related Work	22
21 Conclusion	23

1 Introduction

A blockchain is an ordering machine. Execution is a deterministic function of an ordered input tape and an initial state. For financial applications, the contested object is not only *whether* a transaction is included — it is *where* the transaction appears in the sequence.

In a continuous limit order book, time priority is foundational. If two orders rest at the same price, the order that arrived first receives priority. Market makers quote tighter spreads because the venue gives them a stable queue position. Traders can reason about cancels, replaces, liquidations, and fills because the matching engine has a defined notion of time. Without time priority, a maker has no incentive to post a tight quote — any taker flow it attracts can be jumped by a faster, better-informed actor who saw the maker’s quote first.

Most blockchains do not provide this. They provide coarse blocks and discretionary intra-block ordering. A temporary leader can see plaintext transactions before finalizing their order. That leader can then extract value by reordering, delaying, inserting, or censoring. This is not an implementation accident. It is the structural result of giving a proposer discretion over transaction order while letting the proposer see transaction contents.

PoS_q addresses this directly. It defines a sequencing layer where ordering is continuous, auditable, and produced before contents are visible. PoS_q gives the chain an internal notion of time: a continuously advancing VDF-braided tick stream. Each tick commits to a batch of encrypted transaction commitments. The sequencer signs receipts that bind commitments to positions in the stream. Downstream execution follows the stream exactly.

The central rule is simple:

A market-relevant transaction must be ordered before its market-relevant contents are known.

This removes last-look. The sequencer cannot see a trade, decide whether it is toxic, and then assign it a favorable or unfavorable position. By the time plaintext is available, the receipt, tick, log commitment, and VDF braid already bind the ordering. Any deviation produces compact, slashable evidence.

The remaining problem is censorship before admission. A sequencer can always refuse service. PoS_q makes that strategy ineffective *at scale*. Validators continuously generate encrypted probe transactions drawn from the same observable distribution as user transactions: fixed envelope size, common relay routes, fee payment decoupled from sender identity. These probes are indistinguishable from user flow before the inclusion decision. The sequencer sees encrypted ciphertext, padded metadata, and fee buckets — nothing that identifies whether a packet originates from a market maker, a liquidator, or a validator probe. If the sequencer selectively drops a hidden target, it also drops probes in the same observable bucket. If enough probes fail, validators rotate the sequencer.

PoS_q does not make a metaphysical claim about global packet arrival across the Internet. It enforces FIFO relative to a certified ingress boundary and makes *economically beneficial* deviations infeasible. A sequencer can randomly drop a small fraction of opaque packets. It cannot build a repeatable extraction strategy from transaction contents. Griefing remains possible; targeted MEV does not.

1.1 Contributions

This paper specifies PoS_q as a self-contained sequencing protocol with minimal embedded consensus. Concretely:

1. **Per-user trapdoor timelock encryption.** Each user constructs a timelock ciphertext over an RSA group such that the unlock cost is bounded by sequential VDF work, with a private trapdoor that lets the user (and only the user) reveal cheaply. This removes any need for a decryption committee while keeping unlock under user control.
2. **Certified blind FIFO.** Transactions are admitted as encrypted commitments. The sequencer issues signed receipts or signed rejections during narrow inclusion windows before transaction contents unlock.
3. **VDF-braided timechain.** Transaction batch roots are braided into a continuously advancing RSA-group VDF state, so that the ordered log becomes part of the sequential-time history.
4. **Validator-indistinguishable polling.** Validators submit probe commitments using fixed-size envelopes, anonymous relay paths, and fee tickets paid by accounts unrelated to the validator identity. Probes are indistinguishable from user commitments until after the inclusion deadline. We give a statistically grounded probe rate.
5. **Minimal sequencing consensus.** A purpose-built BFT consensus (which we call *ContSeq*) finalizes prefixes of the PoSq stream in batches, handles leader rotation on objective fault, liveness failure, or scheduled epoch end, and operates under standard $\geq 80\%$ honest-stake assumptions consistent with modern BFT designs.
6. **Objective fraud proofs.** Equivocation, reordering, omission of receipted commitments, invalid VDF transitions, and invalid log transitions produce compact evidence verifiable in constant or logarithmic work.
7. **Deterministic gap handling.** Invalid, expired, malformed, or unrevealed commitments occupy their original positions and cannot be compressed to manufacture reorderings.

2 Design Goals

PoSQ is designed for latency-sensitive markets and other applications where ordering is economically meaningful.

2.1 FIFO Relative to Certified Ingress

If transaction A is admitted before transaction B , and the separation exceeds the configured tick resolution, A must appear before B in the canonical sequence. If A is invalid, it becomes a deterministic gap and B does not move forward to fill it. If A is admitted and later omitted, the sequencer is slashable.

2.2 No Last-Look

The sequencer must commit to ordering before learning market-relevant transaction contents. A market order, cancel, liquidation, or replace cannot be inspected before its queue position is fixed. This is the property a centralized exchange gives by construction (the matching engine and the order receiver are the same trusted system). PoSq reproduces it on a permissionless substrate by enforcing that the receiver cannot decrypt before committing.

2.3 No Profitable Selective Censorship

A sequencer cannot target specific hidden transactions. Validator probes are indistinguishable from user flow. Any censorship policy expressible over pre-admission observables also affects probes in the same bucket. Above the loss tolerance, validators rotate the sequencer; below it, the residual strategy is bounded random loss. Griefing remains possible; profitable targeted extraction does not.

2.4 Continuous Ordering

PoSq produces a continuous tick-indexed transaction stream. The sequencing layer does not wait for full consensus to order every transaction. ContSeq finalizes prefixes of the stream in batches at a coarser cadence; the receipt-and-tick layer produces ordering at sub-millisecond cadence.

2.5 Objective Accountability

Objective faults are slashable. Availability failures cause rotation. The distinction matters: slashing requires compact evidence of misbehavior; rotation only requires the protocol to conclude that the sequencer is no longer providing acceptable service.

3 System Model

3.1 Participants

Clients. Clients submit transactions. For market-moving actions, clients submit timelock-encrypted commitments. The plaintext intent contains chain ID, market ID, account nonce, operation type, fee authorization, expiry, and client signature. The ciphertext is padded to a fixed envelope size and tagged with an approved metadata bucket. Each client controls a private trapdoor that allows cheap reveal of its own ciphertext (Section 5).

Sequencer. The sequencer is the unique leader for a sequencing epoch. It receives encrypted commitments, assigns them to ticks, issues receipts, advances the PoSq timechain, and broadcasts tick records.

The sequencer may be Byzantine: drop messages, delay receipts, equivocate, produce invalid VDF transitions, publish inconsistent logs, or assign commitments to incorrect positions. PoSq makes each of these behaviors either slashable or rotation-triggering.

Validators. Validators verify the PoSq stream, generate indistinguishable probe traffic, aggregate probe-failure reports, run ContSeq to finalize stream prefixes and rotate sequencers, and verify fraud proofs.

Relayers. A permissionless set of relay endpoints accept envelopes from any sender and forward them to the sequencer ingress. Relayers do not see plaintext. Their role is to make the network-layer source of an envelope independent of the economic identity of the transaction. Validators use the same relay set as users when probing.

Watchers. Any party can monitor the stream and submit fraud proofs.

3.2 Network Model

The network is partially synchronous. After global stabilization time, honest messages are delivered within a bounded delay δ . PoSq does not assume equal client latency. It enforces order relative to the protocol ingress boundary — specifically, the moment a valid commitment is admitted by the sequencer.

3.3 Threat Model

The adversary may control the sequencer, a minority of validators (up to 20% of stake; see Section 16), a subset of clients, a subset of relayers, and network scheduling subject to synchrony assumptions. The adversary may submit malformed commitments, flood ingress, refuse to reveal its own commitments, attempt to identify valuable user flow through metadata, and selectively drop traffic.

The adversary cannot forge signatures, find hash collisions, factor large RSA moduli or compute discrete logs in unknown-order groups within the configured safety margin, or compute VDF outputs faster than the sequential-work bound.

4 Time, Ticks, and Windows

PoS defines time as a sequence of ticks. Let Δ be the target tick duration. We recommend $\Delta = 200\mu s$ as a baseline (yielding 5,000 ticks per second), with deployments tunable in the range $100\mu s$ to $1ms$ depending on network conditions and hardware.

A tick has:

- a VDF state x_t ;
- a log commitment C_t ;
- a batch of admitted encrypted commitments B_t ;
- a payload root $\text{root}(B_t)$;
- a tick record signed by the sequencer.

The sequencer runs the VDF continuously. Each tick extends the timechain and binds the current transaction batch into the next VDF input.

4.1 Inclusion Window

An inclusion window is the short interval during which an encrypted commitment must receive one of three responses:

1. an admission receipt;
2. a signed rejection (with a stated reason);
3. a signed full-window receipt.

No silent outcome is valid. A commitment that receives no response by the deadline is a *miss*. Misses on validator probes count toward rotation. The inclusion window must close before the commitment can be cheaply unlocked by anyone other than the holder of the trapdoor:

$$T_{\text{unlock-public}} > T_{\text{admit}} + T_{\text{commit}} + \mu, \quad (1)$$

where μ is a safety margin against early unlock. This inequality applies to public unlock via the VDF path. Trapdoor unlock by the user is fast by construction (Section 5).

4.2 Reveal Delay

A commitment admitted at tick t becomes executable only after a fixed reveal delay of r ticks. At tick T , downstream execution may process tick $T - r$. If reveal material is not available by then, the entry becomes a deterministic gap. We recommend $r = 10$ ticks ($2ms$ at $\Delta = 200\mu s$) as a baseline. The delay must be long enough for honest clients to reveal under normal conditions and short enough to keep market-action latency competitive with centralized venues.

This rule prevents rollback and prevents one unrevealed transaction from stalling the market.

5 Per-User Trapdoor Timelock Encryption

The cryptographic core of PoSq’s no-last-look property is a timelock encryption scheme with two unlock paths:

- **Slow public path** via sequential VDF work, taking time $\geq T_{\text{unlock-public}}$ for any party.
- **Fast trapdoor path** for the user who constructed the ciphertext, taking constant time.

5.1 Construction

We use Rivest–Shamir–Wagner timelock puzzles [1] instantiated over an RSA group, with the encrypting user playing the role of trusted setup for their own ciphertext.

Let $N = pq$ be an RSA modulus chosen by the user, with p, q safe primes. The user knows the trapdoor $\phi(N) = (p - 1)(q - 1)$. To encrypt intent i with public unlock time T :

1. Pick a random $a \in \mathbb{Z}_N^*$.
2. Compute the long way (or, using the trapdoor, fast):

$$b = a^{2^T} \bmod N. \tag{2}$$

A party without $\phi(N)$ must perform T sequential squarings. The user computes $e = 2^T \bmod \phi(N)$ in constant time, then $b = a^e \bmod N$.

3. Derive a key $k = H(b)$ and encrypt the intent: $c = \text{AEAD}_k(i \parallel \sigma_i)$.
4. The ciphertext is $\text{ct} = (N, a, T, c)$.

The protocol fixes a target unlock duration and a calibration of squarings per second on reference hardware, jointly determining T .

5.2 Why This Works for PoSq

The construction has three properties that make it the right primitive here:

1. **No decryption committee.** Many timelock-encrypted-mempool designs use threshold decryption, introducing a committee whose corruption breaks confidentiality. PoSq has no such committee. The trapdoor is held only by the originating user.
2. **Asymmetric unlock cost.** The user can reveal cheaply at any time after admission. An adversary attempting to peek before $T_{\text{unlock-public}}$ must perform T sequential squarings, which by the RSW assumption cannot be parallelized.

3. **Self-contained reveal.** Reveal does not require interaction with a committee or with the sequencer. The client publishes the plaintext, signature, and a short proof that the plaintext was produced from (N, a, T, c) via the trapdoor unlock. Because the user precomputes b via $\phi(N)$, the reveal is a constant-time operation client-side.

5.3 Reveal Procedure

To reveal an admitted commitment at tick t position j , the user publishes:

$$\text{Reveal}_{t,j} = (t, j, b, i, \sigma_i, \pi_{\text{open}}, \text{merkle_path}), \quad (3)$$

where π_{open} is a proof that $\text{AEAD}_{H(b)}^{-1}(c) = i \parallel \sigma_i$ for the c inside the admitted ciphertext. Since AEAD decryption is deterministic given the key, π_{open} is implicit: any verifier can recompute $H(b)$ and try the decryption.

A verifier accepts the reveal iff:

1. the Merkle path opens under $\text{root}(B_t)$;
2. the AEAD decryption with key $H(b)$ succeeds and produces $i \parallel \sigma_i$;
3. the client signature verifies under the account public key in i ;
4. i has not expired.

5.4 Threat-Model Implications

The RSW assumption states that for a randomly chosen a in a group of unknown order, computing $a^{2^T} \bmod N$ requires T sequential squarings. Because each user generates their own modulus, an adversary who breaks one user's modulus does not break others. The cost of factoring an RSA-2048 modulus exceeds the cost of running the VDF for any reasonable T by many orders of magnitude, so in practice the bottleneck for an adversarial sequencer is the sequential work, not factoring.

A user who picks a weak modulus only harms themselves: their own ciphertext may unlock early. A protocol-level wallet implementation should generate moduli using standard safe-prime procedures.

6 Transaction Format and Metadata Discipline

Payload encryption alone is insufficient. Metadata can reveal strategy. PoSq treats metadata minimization as a protocol rule, not as user advice.

A market-moving commitment has the form:

$$m = (\text{epoch}, \text{bucket}, \text{fee_ticket}, \text{ct}, \text{commit_hash}). \quad (4)$$

The commitment hash is:

$$\text{commit_hash} = H(\text{epoch} \parallel \text{ct} \parallel \text{client_commit_nonce}). \quad (5)$$

Fixed envelope. Every market envelope is padded to a single fixed size S at the wire level. There is exactly one valid envelope size for market-action commitments. We recommend $S = 4096$ bytes as a deployment baseline; the choice is bounded below by the largest plausible plaintext intent plus AEAD overhead and the timelock parameters.

Bucket. The visible bucket is an approved metadata class drawn from a small protocol-defined set. It encodes only what the sequencer needs for capacity management: market group, ingress path class, and at most a coarse fee class. It must *not* encode side, price, exact size, account identity, liquidation target, cancel target, or strategy-bearing payload data.

Fee tickets. Fees are quantized to a small number of fee-ticket denominations (we recommend 4–8 classes spanning the practical range). Fee tickets are bearer instruments purchased in advance; they are not bound to a specific account at admission time. This decouples the economic identity of the fee payer from the economic identity of the transaction sender. A market maker’s bot can submit envelopes whose fee tickets were purchased through a relay-mediated pool, indistinguishable from any other envelope’s fee.

Relay-decoupled sender. The network-layer source of an envelope is a relayer, not the user’s economic identity. Validators use the same relay set when probing.

A transaction that violates metadata policy is invalid at ingress.

7 PoSq Admission

7.1 Commit Submission

A client constructs a signed plaintext intent:

$$i = (\text{tx}, \text{chainID}, \text{market}, \text{nonce}, \text{fee}, \text{expiry}). \quad (6)$$

The client signs:

$$\sigma_i = \text{Sig}_{sk_i}(H(i)). \quad (7)$$

The client encrypts $i \parallel \sigma_i$ using its trapdoor timelock (Section 5):

$$\text{ct} = \text{TimelockEnc}_{(N,a,T)}(i \parallel \sigma_i). \quad (8)$$

The client submits, via a relayer:

$$\text{Commit} = (\text{epoch}, \text{bucket}, \text{fee.ticket}, \text{ct}, H(\text{ct})) \quad (9)$$

padded to envelope size S .

7.2 Admission Receipt

For every accepted commitment, the sequencer assigns a tick t and a position j inside that tick batch. The sequencer returns:

$$\rho = \text{Sig}_{\text{seq}}(\text{epoch}, t, j, H(\text{ct}), \text{bucket}, x_t, C_t, d_{j-1}, d_j). \quad (10)$$

The receipt-chain digest is:

$$d_j = H(\text{epoch} \parallel t \parallel j \parallel H(\text{ct}) \parallel \text{bucket} \parallel d_{j-1}). \quad (11)$$

The receipt is the client’s ordering certificate. It states that this encrypted transaction owns position j in tick t and chains to the previous receipt in the same tick. Two receipts with inconsistent d_j chains are direct evidence of within-tick equivocation.

7.3 Signed Rejection

A rejection has the form:

$$\rho^{\text{rej}} = \text{Sig}_{\text{seq}}(\text{epoch}, H(\text{ct}), \text{reason}, \text{bucket}, \text{local_time}, C_t). \quad (12)$$

Valid reasons include: malformed commitment, invalid fee ticket, duplicate commitment, late window, full window. False signed rejections are objective evidence when the rejected commitment is syntactically valid and the stated condition is false.

7.4 Full-Window Receipt

If capacity is exhausted, the sequencer issues:

$$\rho^{\text{full}} = \text{Sig}_{\text{seq}}(\text{epoch}, \text{bucket}, \text{window}, \text{capacity}, C_t). \quad (13)$$

Full-window receipts prevent silent overload and make congestion measurable. A sequencer cannot claim arbitrary fullness while publishing empty capacity in the same bucket. Full-window responses are issued in receipt order over arrived envelopes; the same indistinguishability that protects drops also protects which envelopes get full-window'd.

Resubmission. A client receiving ρ^{rej} for a transient reason or ρ^{full} may resubmit with a fresh nonce in a subsequent inclusion window of the same epoch. The receipt-chain digest d_j is per-(epoch, tick, position); it does not constrain resubmission across windows.

8 Tick Batches and Canonical Order

Each tick contains a bounded batch B_t :

$$B_t = (e_{t,1}, e_{t,2}, \dots, e_{t,n_t}), \quad (14)$$

where each entry is:

$$e_{t,j} = (H(\text{ct}), \text{bucket}, \rho_{t,j}). \quad (15)$$

Canonical order is lexicographic by:

1. tick t ;
2. position j ;
3. for true within-tick concurrency (entries with no receipt-order relation), commit-hash sort.

Within a tick, the default order is receipt order. The receipt-chain digest d_j provides this directly: any verifier can reconstruct the receipt sequence from the chain. PoSq does not sort by transaction hash as a primary ordering rule, because hash ordering is grindable when an adversary knows plaintext; here the entries being sorted are ciphertexts under fresh randomness, so hash sort is a defensible tie-breaker. We accept this for the within-tick concurrent case because Δ is small enough ($200\mu s$ baseline) that genuine concurrency is rare and economically marginal, and the alternative (future-randomness tie-breaking) introduces complexity that is not justified at this resolution.

The canonical stream is:

$$B_0, B_1, B_2, \dots \quad (16)$$

Downstream execution consumes that stream exactly.

9 Braided VDF Timechain

A plain VDF clock is not enough. A sequencer could compute future VDF states in advance and later attach transaction batches to a precomputed timeline. PoSq prevents this by braiding the transaction log into the VDF input, so the sequential-work history depends on the actual batch sequence.

9.1 Construction

We use a Wesolowski VDF [2] over an RSA group with a publicly verifiable proof. Let G_{seq} be a fixed RSA group with modulus N_{seq} generated by a class-group setup or a one-shot trusted ceremony at protocol genesis (the modulus need not be the same as user moduli). Let $x_t \in G_{\text{seq}}$ be the VDF state at tick t , and let $\text{root}(B_t)$ be the Merkle root of the tick batch.

Define:

$$m_t = H(\text{root}(B_t) \parallel t \parallel pk_{\text{seq}}). \quad (17)$$

The next VDF input is:

$$\tilde{x}_t = H_G(x_t \parallel m_t), \quad (18)$$

where H_G is a hash-to-group function for G_{seq} .

The next VDF state is:

$$x_{t+1} = \tilde{x}_t^{2^{T_\Delta}} \bmod N_{\text{seq}}, \quad (19)$$

where T_Δ is the number of squarings calibrated to wallclock Δ on reference hardware.

The Wesolowski proof π_t is a single group element verifiable in $O(\log T_\Delta)$ work. Proofs are batched across tick ranges so validators do not verify a standalone proof per tick.

The log commitment evolves as:

$$C_{t+1} = H(C_t \parallel \text{root}(B_t) \parallel x_{t+1} \parallel t + 1). \quad (20)$$

The sequencer broadcasts:

$$\text{TickRecord}_t = (\text{epoch}, t, x_t, \text{root}(B_t), C_t, x_{t+1}, C_{t+1}, \pi_t, \text{DA}_t, \sigma_{\text{seq}}), \quad (21)$$

where:

$$\sigma_{\text{seq}} = \text{Sig}_{\text{seq}}(\text{epoch} \parallel t \parallel \text{root}(B_t) \parallel x_{t+1} \parallel C_{t+1} \parallel \text{DA}_t). \quad (22)$$

9.2 Why Braiding Prevents Pre-computation

The intended property is: *the sequencer cannot publish x_{t+1} before knowing $\text{root}(B_t)$* . This holds under two assumptions:

Assumption 1 (Sequentiality of squaring in G_{seq}). *For a uniformly random $\tilde{x} \in G_{\text{seq}}$ where G_{seq} has unknown order, computing $\tilde{x}^{2^{T_\Delta}}$ requires at least T_Δ sequential squarings.*

Assumption 2 (Hash-to-group is preimage-binding). *H_G is modeled as a random oracle into G_{seq} . An adversary cannot find (x', m') such that $H_G(x' \parallel m')$ equals a target value with probability better than negligible.*

Under these, suppose the sequencer publishes a TickRecord_t at wallclock τ such that $\text{root}(B_t)$ was finalized at time $\tau - \epsilon$ for $\epsilon < \Delta$. Then either the sequencer started the squaring chain for x_{t+1} before $\text{root}(B_t)$ existed (impossible by hash-to-group binding, since $\tilde{x}_t$ depends on m_t which depends on $\text{root}(B_t)$), or it completed T_Δ squarings in less than Δ wallclock, contradicting sequentiality.

The Wesolowski proof binds the claimed x_{t+1} to the claimed input $\tilde{x}_t$, so a sequencer cannot present an inconsistent state.

9.3 Verification

A verifier checks the tick stream by verifying, for each tick: the Wesolowski proof, the braid hash, the log commitment update, the sequencer signature, and the availability of B_t via DA_t . Because transaction batches are braided into the VDF input, rewriting the past requires rebuilding the sequential-work history. Because tick records are signed, presenting two different histories produces compact equivocation evidence.

10 Data Availability

A signed tick root without data is not enough. Validators must be able to reconstruct the tick batch.

Each tick record includes a data-availability reference DA_t . In the simplest deployment, the full tick batch is gossiped with the tick record. In a higher-throughput deployment, batches are erasure-coded across validators and a DA certificate is required for finalization.

The voting rule is:

No data, no vote.

A tick that references unavailable PoSq data is not eligible for finalization by ContSeq.

11 Validator-Indistinguishable Polling

The remaining censorship problem is pre-admission refusal. PoSq handles it with validator polling. Validators continuously submit encrypted probe commitments through the same ingress paths, with the same envelope size, fee-ticket distribution, and relayer set as users. The sequencer cannot tell whether a given encrypted packet is a user order, a validator probe, or ordinary low-value flow.

11.1 Probe Generation

Each validator uses a private schedule derived from a VRF:

$$\text{seed}_{v,e,k} = \text{VRF}_{sk_v}(\text{epoch} \parallel k). \quad (23)$$

The seed determines, for each probe k in epoch e :

- the wallclock at which to send;
- the metadata bucket;
- the relayer to use;
- the fee ticket;
- the fee-ticket source account (drawn from a protocol-controlled pool funded by validator stake);
- the timelock-encrypted probe payload, drawn from a protocol-defined family of valid no-op transactions.

Probe payloads are valid transactions: protocol-owned heartbeats or controlled self-cancels designed to have zero market impact. They pay the same admission fees and use the same envelope size, padding, and bucket discipline as user transactions.

The probe identity (i.e., that this specific commitment was a probe) is revealed by VRF proof only after the inclusion deadline.

11.2 Probe Success and Failure

A probe *succeeds* if the validator receives, before the deadline, one of:

- an admission receipt;
- a valid signed rejection;
- a valid full-window receipt.

A probe *fails* if the sequencer is silent past the deadline, replies late, gives an invalid rejection, or gives a false full-window response (one whose stated capacity contradicts published capacity for the same bucket and window).

11.3 Per-Bucket Polling and Statistical Calibration

Polling is measured per bucket. A global failure rate is insufficient: a sequencer must not be able to censor one market, one region, one relay, one fee class, or one size class while keeping global failure low.

A bucket is defined by deployment policy. Typical bucket dimensions:

$$\text{bucket} = (\text{market_group}, \text{fee_class}, \text{ingress_path}, \text{region}). \quad (24)$$

Probe rate calibration. Let λ be the tolerated random-loss rate in a bucket. We need enough probes per polling window so that an attacker dropping users at rate $\lambda + \epsilon$ for some economically meaningful ϵ is detected with high probability before significant value can be extracted.

Suppose N_p probes are sent in a polling window and the sequencer drops at rate $\lambda + \epsilon$. The expected number of probe failures is $N_p(\lambda + \epsilon)$ with standard deviation $\sqrt{N_p(\lambda + \epsilon)(1 - \lambda - \epsilon)}$. To distinguish $\lambda + \epsilon$ from λ at confidence corresponding to z standard deviations, we require:

$$N_p \geq \frac{z^2 \lambda(1 - \lambda)}{\epsilon^2}. \quad (25)$$

Setting $\lambda = 0.05$, $\epsilon = 0.02$ (we want to detect a drop rate of 7% versus the tolerated 5%), and $z = 3$ (false-positive rate $\approx 0.13\%$ per window per bucket), we obtain:

$$N_p \geq \frac{9 \times 0.05 \times 0.95}{0.0004} \approx 1,069 \text{ probes per window per bucket}. \quad (26)$$

For a polling window of 1 second and a deployment with, say, 32 valuable buckets, this is $\sim 34,000$ probes per second across the validator set — a small fraction of total throughput at the recommended $\Delta = 200\mu s$ tick rate. With 100 validators, each validator emits roughly 340 probes per second. We recommend tightening to $\epsilon = 0.01$ for high-value buckets, which raises N_p to $\sim 4,275$ per window per bucket.

Bucket coverage requirement. If a bucket carries valuable flow, it must carry validator probes meeting the calibration above. A bucket without probes is not eligible for high-value market flow; the deployment policy must reject commitments to such buckets at the protocol layer.

11.4 Probe Verifiability

To prevent a colluding validator coalition from inflating failure rates with malformed probes (a denial-of-service-on-sequencer vector), every probe must be *verifiable as well-formed* after reveal. A miss report MR_v is only counted toward an Availability Rotation Certificate if accompanied by:

- the VRF proof binding the probe to the validator’s schedule;
- the timelock parameters and post-deadline opening of the probe payload;
- evidence of conforming envelope size, bucket, and fee ticket.

Probes that fail to verify are discarded, and a validator submitting a threshold of unverifiable probes can itself be slashed for protocol misbehavior.

11.5 Availability Rotation Certificate

Validators aggregate verifiable miss reports:

$$\text{MR}_v = \text{Sig}_v(\text{epoch}, \text{bucket}, \text{window}, \text{probe_commitment}, \text{deadline}, \text{status}, \text{vrf_proof}, \text{open}). \quad (27)$$

An *Availability Rotation Certificate* (ARC) forms when validators representing at least θ of voting stake report verified probe failures above λ in the same bucket and polling window. We set $\theta = 20\%$ as the baseline (Section 16).

An ARC rotates the sequencer at the next ContSeq rotation boundary (or immediately if the deployment enables emergency rotation). Rotation is not slashing: it is a liveness failover. Objective faults are slashable; availability failure removes the sequencer from the role.

To prevent rotation cycling by a malicious 21% coalition, the protocol enforces an ARC cooldown: after a rotation, the rotating coalition cannot trigger another ARC against the next sequencer for at least W ticks, and consecutive ARCs by overlapping coalitions face progressive stake-bond requirements.

11.6 Effect of Polling

Without polling, a sequencer can silently drop selected traffic and claim it never arrived. With indistinguishable polling, the sequencer cannot know which packets are probes. If it drops a hidden class of user transactions, it drops probes in the same observable bucket. If it drops enough to matter, it triggers rotation. If it stays below the threshold, the residual attack is bounded random loss.

A sequencer can drop 5% of opaque packets randomly. It cannot reliably drop “the profitable transaction,” “the toxic cancel,” “the liquidation,” or “the maker’s order” before reveal. Those labels are hidden. The sequencer sees encrypted traffic classes, not trading intent.

12 Selective Censorship Collapse

This section states the central security argument.

Let b be a metadata bucket. Let U_b be user commitments in bucket b and P_b be validator probes in bucket b . By construction of fixed envelope size, common relayer set, decoupled fee tickets, and protocol-defined valid payloads, before the inclusion deadline U_b and P_b are drawn from the same observable distribution $\mathcal{D}_b$.

The sequencer chooses an admission policy:

$$A : \text{view} \rightarrow \{\text{admit}, \text{reject}, \text{drop}\}, \quad (28)$$

where view includes only pre-admission observables: bucket, envelope size, fee class, ingress path, timing pattern, ciphertext bits.

The sequencer does not observe plaintext, side, price, account strategy, liquidation target, cancel target, or transaction type.

Claim 1 (Selective censorship collapse). *Against a sequencer that cannot distinguish validator probes from user commitments before the inclusion decision, any strategy that selectively censors hidden transaction content either triggers rotation or degenerates into bounded random loss.*

Proof sketch. The sequencer’s admission policy can depend only on its observable view. By indistinguishability, U_b and P_b in the same bucket have the same observable distribution before the inclusion deadline. A policy that drops a hidden class of user commitments must act through visible correlates. Those correlates also apply to probes in the same bucket. The probe failure rate therefore tracks the user failure rate for that observable class. If the failure rate exceeds λ , validators form an ARC and rotate the sequencer; the calibration of Section 11.3 ensures detection within the polling window with overwhelming probability. If the failure rate remains below λ , the effect is bounded random loss within the bucket. Since hidden transaction content is unavailable, the sequencer cannot reliably target economically valuable transactions. $\square$

Griefing vs. extraction. Claim 1 does not say censorship is impossible. A sequencer can refuse service to all envelopes (denial of service), drop a small random fraction (griefing within tolerance), or drop predictably enough to trigger rotation (one-shot griefing). What it cannot do is build a *repeatable, profitable* extraction strategy against hidden content. The protocol’s economic discipline requires that the sequencer’s bond and forfeitable rewards exceed the maximum extractable value available during the rotation latency (Section 15).

This is the effective FCFS property needed by a market. The protocol does not promise perfect packet delivery. It promises that admitted order is canonical, that hidden-content targeting is unavailable, and that scale censorship is detected by the same validators that secure the sequencing layer.

13 Fraud Proofs

PoS_q fraud proofs are compact. A challenger does not need to reproduce the entire epoch, only the inconsistent commitments.

13.1 Equivocation Proof

Two signed tick records, receipts, or stream heads that are inconsistent for the same epoch and tick:

$$\Pi_{\text{eqv}} = (M, M'), \quad (29)$$

where both verify under pk_{seq} , refer to the same position or prefix, and commit to different values.

13.2 Reorder Proof

Two valid receipts and a stream opening showing that the later receipt appears earlier:

$$\Pi_{\text{reorder}} = (\rho_a, \rho_b, \text{path}_a, \text{path}_b, \text{TickRecord}_t), \quad (30)$$

with $\text{pos}(a) < \text{pos}(b)$ by receipt-chain digest d_j , but b preceding a in the committed stream.

13.3 Omission Proof

A valid admission receipt and evidence that the corresponding entry is absent from the assigned tick batch:

$$\Pi_{\text{omit}} = (\rho, \text{TickRecord}_t, \text{DA}_t, \text{noninclusion_proof}). \quad (31)$$

In the simplest design, validators reconstruct the full tick batch from DA_t and check that the receipted entry is missing.

13.4 Invalid VDF Transition Proof

$$x_{t+1} \neq \tilde{x}_t^{2^{T\Delta}} \bmod N_{\text{seq}}, \quad \tilde{x}_t = H_G(x_t \parallel m_t). \quad (32)$$

The proof includes the tick record and a Wesolowski verification failure.

13.5 Invalid Log Transition Proof

$$C_{t+1} \neq H(C_t \parallel \text{root}(B_t) \parallel x_{t+1} \parallel t + 1). \quad (33)$$

13.6 Late Insertion Proof

An entry appears in a tick despite not having a valid receipt for that tick and position.

13.7 Invalid Gap Proof

An entry marked as unrevealed or invalid actually has a valid reveal before the reveal deadline.

14 ContSeq: Minimal Sequencing Consensus

PoS_Q ordering is produced at tick cadence. Finality, leader rotation, and slashing decisions require agreement among validators. We define *ContSeq*: a minimal Byzantine-fault-tolerant consensus protocol whose only job is to (i) finalize prefixes of the PoS_Q stream, (ii) rotate sequencers, and (iii) record slashing decisions.

14.1 Scope

ContSeq is deliberately narrow. It does not order user transactions itself — PoS_Q does that. It does not execute transactions — a downstream execution layer does that. ContSeq’s input is the sequence of signed tick records produced by the sequencer; its output is a chain of finalized prefixes and a record of leader changes.

14.2 Validator Set and Honesty Assumption

ContSeq runs over a stake-weighted validator set. Following modern BFT designs, the safety threshold assumes at least 80% honest stake; the protocol tolerates up to 20% Byzantine stake without violating safety. This is consistent with the rotation threshold θ of Section 11: a 20% minority can rotate a sequencer for liveness reasons, and an 80% majority is required to finalize a stream prefix.

Assumption 3 (ContSeq honest stake). *At any point during a ContSeq epoch, at least 80% of voting stake is honest and online.*

14.3 Finalization

Validators run a HotStuff-style three-phase commit [5] on *batches* of tick records. A finalization batch covers K consecutive ticks (we recommend $K = 500$, giving a finalization cadence of $100ms$ at $\Delta = 200\mu s$). At each batch boundary, validators:

1. verify the VDF proofs for the ticks in the batch;
2. verify data availability for each tick;
3. verify log-commitment continuity;
4. check for outstanding fraud proofs against the batch;
5. check for ARCs against the current sequencer.

A batch is finalized iff 80% of stake votes for it. Once finalized, the batch is irreversible under the honest-stake assumption.

14.4 Leader Rotation

A sequencer is rotated for any of the following reasons:

Objective fault (slashable). A valid fraud proof against the current sequencer triggers immediate rotation and slashing of the sequencer’s bond.

Availability failure. A valid ARC (Section 11) triggers rotation at the next batch boundary or immediately if the deployment enables emergency rotation. The sequencer’s bond is forfeit if the ARC is sustained across multiple consecutive windows; a single transient ARC may incur a smaller penalty per deployment policy.

Scheduled rotation. The sequencer rotates every 24 hours of wallclock by default. Scheduled rotation is not punitive — it limits the time-window over which any single sequencer accumulates information about user flow patterns.

Liveness failure. If the sequencer fails to produce tick records for L consecutive ticks (we recommend $L = 50$, i.e., $10ms$ at baseline parameters), validators may form a liveness-failure certificate equivalent to an ARC.

When a sequencer rotates, the new sequencer continues from the latest finalized stream state $(x_t, C_t, \text{epoch} + 1)$. The new epoch inherits the canonical prefix and begins producing new ticks from that state. Outstanding inclusion windows that opened under the rotated sequencer are reissued by the new sequencer; clients with admitted but unfinalized commitments retain their receipts, and the new sequencer is bound to honor the canonical prefix or face omission proofs.

14.5 Why a Narrow Consensus Is Appropriate

A general-purpose L1 consensus does too much for what PoSq needs. It orders user transactions, executes them, and commits state, all in the same critical path. PoSq’s design separates these concerns: ordering happens at PoSq tick cadence, execution can happen on a derived stream, and ContSeq finalizes the ordering layer in coarser batches without standing between the user and the sequencer in the latency-critical path.

This separation has two effects. First, finality cadence ($100ms$ batches) does not constrain ordering cadence ($200\mu s$ ticks). Second, the consensus layer becomes simpler: it agrees on prefixes

of an already-canonical signed stream, not on which transactions to include and in what order. The Byzantine-tolerance threshold and proof complexity collapse to standard HotStuff with VDF and DA verification as additional preconditions.

15 Sequencer Bonds and Economic Discipline

The sequencer posts a bond at the start of each epoch. The bond plus forfeitable epoch rewards must exceed the maximum extractable value available during the rotation latency:

$$\text{bond} + \text{epoch_rewards} \geq L_{\text{rot}} \cdot \text{MEV}_{\text{peak}}, \quad (34)$$

where L_{rot} is the upper bound on rotation latency (one ARC formation window plus one ContSeq batch, ~ 1 – 2 seconds in practice) and MEV_{peak} is an estimate of peak extractable value per second under a captured sequencer in the deployed market.

For a perpetual-futures venue with significant volume during volatile periods, MEV_{peak} may be in the tens of thousands of dollars per second. The bond should be sized accordingly. The deployment policy should publish the assumed MEV_{peak} and revisit it as volume grows.

This makes one-shot degradation negative expected value for the sequencer.

16 Threshold Choice

The two key thresholds are the random-loss tolerance λ and the rotation stake threshold θ .

$\lambda = 5\%$ as the targeting boundary. The 5% figure is not a claim that 5% losses are economically negligible — in a perps context, 5% of orders is a meaningful number. It is the rate below which *targeting* becomes statistically infeasible. With N_p probes per window calibrated as in Section 11.3, a sequencer attempting to drop more than $\lambda + \epsilon$ for any economically meaningful ϵ is detected with overwhelming probability. Below λ , the sequencer’s drops are necessarily uncorrelated with payload contents, so they reduce to bounded random loss — which is a grieving cost, not an extraction primitive.

$\theta = 20\%$ as the rotation threshold. We choose $\theta = 20\%$ because the sequencing role is a performance role, not a finality role. A minority of validators should be able to remove a sequencer that is failing to provide service. The dual threshold (80% for finalization, 20% for rotation) is consistent with modern BFT honest-stake assumptions: the protocol tolerates a 20% Byzantine minority for safety, and that same minority is empowered to demand sequencer rotation for liveness.

The asymmetry creates a denial-of-service consideration: a colluding 20% minority could cycle through sequencers by issuing sequential ARCs. The protocol mitigates this with: (i) the ARC cooldown (Section 11), (ii) progressive stake-bond requirements for repeat ARC participants, (iii) the requirement that ARCs be backed by verifiable miss reports (Section 11), and (iv) slashing for validators submitting unverifiable miss reports.

17 Security Analysis

17.1 Ordering Binding

After the sequencer signs a receipt and tick record, it cannot move the transaction without producing inconsistent signed evidence or breaking the log commitment. A receipt binds $(H(\text{ct}), t, j)$. The tick

root binds the batch. The log commitment binds the tick sequence. The VDF braid binds the log into sequential time. Any alternate history diverges in C_t , x_t , or signed tick records.

Lemma 1 (Receipt binding). *Assuming signature unforgeability and hash collision resistance, a sequencer cannot assign the same valid commitment to two different positions without producing slashable evidence (specifically, an equivocation proof).*

17.2 No Plaintext-Based Reordering

Market actions are timelock-encrypted before admission. The reveal delay r exceeds the inclusion window. The trapdoor is held only by the user. Therefore the sequencer commits to position before learning transaction contents.

A sequencer cannot place a trade behind its own back-run after seeing the trade. It cannot inspect a cancellation and delay it. It cannot identify a liquidation target before assigning the liquidation to a tick.

Lemma 2 (No-peek ordering). *Assuming the RSW timelock assumption and Wesolowski VDF soundness, the sequencer cannot condition the position of an honest encrypted commitment on its plaintext contents before the inclusion deadline.*

17.3 No Profitable Selective Censorship

Validator probes are indistinguishable from user flow inside each metadata bucket (fixed envelope, common relays, decoupled fees, protocol-defined payloads). A sequencer that censors hidden content must censor without knowing which packets contain that content. The resulting drop policy hits probes at the same rate.

Above λ , validators rotate the sequencer. Below λ , the strategy is bounded random packet loss, which does not reveal payload, does not change the order of admitted transactions, and does not create a reliable extraction primitive. (Lemma derived from Claim 1 in Section 12.)

17.4 Griefing

Bounded random loss is griefing, and griefing is possible. A sequencer can drop up to λ of all opaque packets before triggering rotation. Per-bucket measurement prevents the sequencer from concentrating those drops on a single market or region, but spread evenly across all buckets, λ losses are unavoidable until the rotation threshold is hit. This is a deliberate design choice: the alternative (zero tolerance) creates rotation thrashing under any genuine network noise.

The economic claim is narrower than “no censorship”: economically beneficial censorship is bounded. A sequencer cannot griefingly extract value, because the targeting precondition is broken.

17.5 Censorship After Receipt

Once a receipt is issued, failure to include the transaction at its assigned position is objective misbehavior. The client presents the receipt; validators check the tick batch via DA_t ; if the entry is absent or misplaced, the sequencer is slashable.

17.6 Gaps

Invalid or unrevealed transactions remain in position as gaps. They do not allow later transactions to move earlier. This prevents a sequencer from using malformed commitments (its own or others’) to manufacture reordering opportunities.

17.7 Adversarial Probes

A 20% validator coalition cannot inflate the probe-failure rate by submitting unverifiable probes: miss reports must include VRF proofs, post-deadline opens, and conformity evidence (Section 11). Validators submitting unverifiable miss reports are themselves slashable. The minimum coalition required to falsely rotate a sequencer therefore must produce well-formed probes deliberately failing in a coordinated bucket, which is observable and addressable through the cooldown and progressive-bond mechanism.

17.8 Sequencer-Affiliated Flow

A sequencer’s own affiliated trading flow must follow the same ingress path, envelope discipline, and metadata policy as all other flow. A deployment may additionally prohibit active sequencer-principal trading during sequencing epochs, by treating any account whose flow correlates with sequencer-affiliated keys as ineligible during the epoch.

18 Performance Model

PoS throughput is:

$$\text{TPS} \approx \text{tick rate} \times \text{batch capacity}. \quad (35)$$

The tick cadence determines ordering resolution. Batch capacity determines throughput. For example:

- $\Delta = 200\mu s$ gives 5,000 ticks per second;
- batch capacity 20 gives 100,000 commitments per second;
- batch capacity 4 gives 20,000 commitments per second.

Receipt latency is dominated by network latency to the sequencer plus inclusion-window response time. Ordering latency is dominated by the next tick boundary ($\leq \Delta$). Reveal latency is dominated by $r\Delta$. VDF proofs are batched over tick ranges.

ContSeq finalization adds $K\Delta$ at the batch boundary plus consensus voting latency. At $K = 500$, $\Delta = 200\mu s$, finalization cadence is $100ms$ plus voting — comparable to fast L1s and uncoupled from the latency-critical ordering path.

19 Deployment Rules

A PoS deployment should enforce the following rules.

Market actions are encrypted. Orders, cancels, replaces, liquidations, and other market-moving actions use trapdoor timelock encryption. Plaintext market actions are not admitted.

Metadata classes are protocol-defined. Users do not choose arbitrary visible metadata. Valid commitments must use the fixed envelope size, an approved fee bucket, and an approved ingress path class.

Probes cover valuable buckets. Every valuable traffic bucket receives validator probes meeting the calibration of Section 11.3. A bucket without probes is not eligible for high-value market flow.

Parameter	Recommended Value
Tick duration Δ	$200\mu s$
Inclusion response deadline	1–2 ticks
Reveal delay r	10 ticks ($2ms$)
Probe random-loss tolerance λ	5%
Probe detection margin ϵ	1%–2%
Rotation stake threshold θ	20%
ContSeq honest-stake assumption	80%
ContSeq batch size K	500 ticks ($100ms$)
Liveness-failure tick count L	50 ticks ($10ms$)
Scheduled rotation interval	24h
Envelope size S	4,096 bytes
Fee-ticket denominations	4–8 classes
Per-bucket probes per window (1s)	1,000–4,500

Table 1: Representative PoSq deployment parameters.

Rotation is fast. Availability failure removes the sequencer at the next batch boundary or immediately. Objective misbehavior slashes it.

Sequencer-affiliated flow follows the same path. Sequencer-affiliated transactions use the same encrypted ingress path, envelope discipline, and receipt rules as all other flow.

Bonds cover the rotation window. The sequencer bond and forfeitable epoch rewards must exceed peak MEV during the rotation latency (Section 15).

Scheduled rotation every 24h. No sequencer holds the role longer than 24 hours of wallclock. This bounds information accumulation about user flow patterns.

20 Related Work

Fair-ordering protocols. Aequitas and Themis [6, 7] formalize order fairness in Byzantine consensus by aggregating arrival timestamps from honest replicas. PoSq takes a different approach: it preserves a single fast sequencer but makes its ordering commitments blind, auditable, and rotation-policed. The two approaches are complementary — Aequitas-style fairness can be added downstream of a PoSq stream but is not necessary to remove last-look or to bound targeted censorship.

Encrypted mempools. Threshold-decryption mempools (Shutter, Ferveo, and similar) hide transaction contents until after ordering, but rely on a decryption committee whose corruption breaks confidentiality. PoSq’s per-user trapdoor timelock removes the committee: each user is trustless with respect to their own ciphertext, and unlocks (or fails to unlock) on their own.

Verifiable delay functions. Wesolowski and Pietrzak VDFs [2, 3] provide the sequential-work primitive PoSq braids into its timechain. Solana’s Proof of History [4] introduced the idea of a sequential clock for blockchain ordering; PoSq braids transaction batches into the delay chain, so

the transaction log itself becomes part of the time history — closing the precomputation gap that PoH does not address.

Shared sequencers. Astria, Espresso, and similar shared-sequencer designs decouple ordering from execution at the rollout level. PoSq is similar in scope but adds three properties most shared sequencers do not have today: blind admission via timelock, indistinguishable polling for censorship resistance, and a tight consensus dedicated to sequencing finality. Flashbots SUAVE and Chainlink FSS occupy related design space, with different choices about who holds the sequencer role and how MEV is internalized.

MEV research. A large literature [8, 9] documents how plaintext transaction visibility and discretionary proposer ordering create frontrunning, back-running, and priority gas auctions. PoSq removes the proposer’s ability to condition ordering on hidden transaction contents and removes the proposer’s ability to selectively censor at scale — the two preconditions for the dominant MEV strategies.

21 Conclusion

PoSq treats ordering as a first-class protocol invariant. It gives a blockchain a canonical, VDF-braided, tick-indexed transaction stream. Market transactions are encrypted under per-user trapdoor timelocks before admission. The sequencer issues signed receipts before plaintext unlocks. Validators poll the sequencer with envelopes indistinguishable from user flow. ContSeq, a minimal sequencing-only consensus, finalizes prefixes of the stream and rotates leaders on fault, liveness failure, or schedule.

A sequencer that reorders is slashable. A sequencer that selectively censors is rotated. A sequencer that stays below rotation thresholds is reduced to bounded random packet loss. Griefing remains possible at small scale; profitable targeted MEV does not.

This is the practical ordering primitive needed for on-chain markets. PoSq gives downstream execution a credible notion of time and preserves price-time priority without requiring full consensus to order every transaction. The sequencing layer produces a verifiable stream. Execution and finality consume that stream.

POSq enables an efficient economic approximation of FCFS transaction ordering in Byzantine settings.

References

- [1] Rivest, R.; Shamir, A.; Wagner, D. *Time-lock puzzles and timed-release crypto*. MIT/LCS/TR-684, 1996.
- [2] Wesolowski, B. *Efficient verifiable delay functions*. EUROCRYPT 2019.
- [3] Pietrzak, K. *Simple verifiable delay functions*. ITCS 2019.
- [4] Yakovenko, A. *Solana: A new architecture for a high performance blockchain*. 2018.
- [5] Yin, M.; Malkhi, D.; Reiter, M. K.; Gueta, G. G.; Abraham, I. *HotStuff: BFT consensus with linearity and responsiveness*. PODC 2019.
- [6] Kelkar, M.; Zhang, F.; Goldfeder, S.; Juels, A. *Order-fairness for Byzantine consensus*. CRYPTO 2020.
- [7] Kelkar, M.; Deb, S.; Long, S.; Juels, A.; Kannan, S. *Themis: Fast, strong order-fairness in Byzantine consensus*. CCS 2023.
- [8] Daian, P.; Goldfeder, S.; Kell, T.; Li, Y.; Zhao, X.; Bentov, I.; Breidenbach, L.; Juels, A. *Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges*. S&P 2020.
- [9] Qin, K.; Zhou, L.; Gervais, A. *Quantifying blockchain extractable value: How dark is the forest?* S&P 2022.